# Presentation script: CacheWise replication and KV cache recovery

This script follows the twelve-slide deck and its 10:00 timing plan. Spend 0:00–5:00 on the CacheWise paper and adapted replication (slides 1–7), then 5:00–10:00 on recovery costs, host methods, limitations and takeaways (slides 8–12). Questions follow the talk. Read the spoken paragraphs; italic instructions are delivery cues and should not be read aloud. Pause briefly after the main numbers and use the slide transitions to check your timing.

The workload graphs on slides 3–4 are reproduced from the original paper; the later serving results are our adapted replication and the separate host study.

## Slide 1 — CacheWise replication and KV cache recovery

**0:00–0:15 · 15 seconds**

*Introduce the question, then move directly to the traces.*

This project studies KV reuse for coding agents under limited GPU memory.

We first examine CacheWise and its replication, then motivate and describe host-cache recovery.

## Slide 2 — What do the CacheWise traces contain?

**0:15–0:40 · 25 seconds**

*Point to the logged events. The next slides show their distributions.*

CATraces records real Claude Code sessions: model requests, tool calls, timings, and token usage.

The released content is redacted, so our replay preserves lengths and prefix growth with synthetic tokens.

The next graphs come from the original paper and show why this workload needs special cache management.

## Slide 3 — CATraces: long inputs, short outputs

**0:40–1:25 · 45 seconds**

*Compare the red CATraces curve with the orange ShareGPT curve in each panel.*

These are the original paper’s prefill and decode distributions. A CDF shows the fraction of requests at or below a token length.

Compare CATraces in red with ShareGPT chat in orange. On the left, the CATraces curve is much farther right: prompts are longer. On the right, it is farther left: outputs tend to be shorter.

Repeatedly processing a large shared prefix can therefore be expensive even when little new text is generated. These are published results, separate from our replay.

## Slide 4 — CATraces differ from traditional chat

**1:25–2:00 · 35 seconds**

*Compare the labeled datasets. In the right-hand plot, CATraces is blue and ShareGPT is purple.*

ShareGPT is the chat baseline here. CATraces has many more turns in the left plot. On the right, follow blue CATraces versus purple ShareGPT: agent contexts grow much larger.

One user task drives repeated model and tool calls, so KV must survive between requests and session completion matters alongside individual response latency.

## Slide 5 — CacheWise predicts when a session returns

**2:00–3:05 · 65 seconds**

*Explain scheduling briefly, then follow the tool-metadata prediction path and remaining-time formula.*

The scheduler serves the request needing the fewest additional KV blocks: required state minus resident state.

For eviction, CacheWise predicts tool-call duration from historical calls. The paper uses TF–IDF representations of arguments and KMeans clusters to group similar calls and their durations.

For a tool still running after elapsed time e, it estimates the remaining time using only durations longer than e. That is the conditional expectation shown here.

It evicts unreferenced KV whose predicted reuse is farthest away and refreshes estimates as time passes. The relative ordering matters. For example, prefer retaining state predicted to return in one second over state predicted to return in thirty seconds.

Our replication uses tool names only because arguments are redacted. That is an adaptation of the paper’s predictor.

## Slide 6 — CacheWise combines scheduling and eviction

**3:05–3:55 · 50 seconds**

*Identify the row and column labels, then the highlighted CacheWise cell and the testbed panel.*

Rows show the scheduler; columns show the eviction policy.

The bottom-right cell is adapted CacheWise: prefix-aware scheduling plus predictive eviction. The top-left is the first-come, first-served LRU baseline. The other cells isolate scheduling or eviction.

The testbed panel lists our Qwen coding model, Blackwell GPU, precision, and cache settings.

N segments are launched as a group. Each continues through its recorded turns, waiting for the previous response and tool delay. Completed segments are not replaced.

Five settings, four policies, and three repeats give sixty runs and 6,384 requests. Our serving-time metric excludes external tool waits.

## Slide 7 — Prefix / LRU has the lowest mean

**3:55–5:00 · 65 seconds**

*Trace the green line. Explain the range bars without reading every number.*

The green prefix-aware LRU line is the scheduling-only ablation. Adapted CacheWise is the purple prefix/predictive curve. The green line has the lowest mean at all three tested loads. Compared with first-come, first-served LRU, the reduction is about 58, 62, and 65 percent as N increases.

It also has the lowest mean in two additional held-out workloads, making five settings out of five.

The ranges show observed minima and maxima from three repeats, not confidence intervals. The load sweep adds segments, so concurrency and composition both change.

These are our adapted replay results, separate from the published workload graphs shown earlier. The purple curve also shows that adding predictive eviction does not improve on prefix-aware LRU in this sweep. We now turn to recovery after eviction.

## Slide 8 — Host restoration: cumulative transfer work

**5:00–6:15 · 75 seconds**

*Begin the second half at 5:00. Explain the stacked operation totals before discussing the separate session metric.*

Saving evicted KV lets us restore it instead of repeating prefill. This chart now shows cumulative operation time across each run.

Blue is saving from GPU to host, and green is restoring from host to GPU. At twenty-eight episodes, they total 7.66 seconds. At thirty-six, they total 23.89 seconds.

These CUDA-event sums include GPU staging and copies. They may overlap model execution, so they are not session latency or a complete measure of management overhead.

The archive does not record isolated recomputation time. We therefore cannot use this chart to give a direct recomputation-to-transfer time ratio.

The sidebar retains a separate result: mean session time improves by ratios of about 9.1 and 4.8 with restoration. That comparison adds thirty-six gibibytes of host KV capacity to the common twenty-four-gibibyte GPU pool.

Each load has one observation. The backing tier is host DRAM; these are not SSD measurements.

## Slide 9 — Preserve evicted KV in host DRAM

**6:15–7:15 · 60 seconds**

*Follow the save arrow to host memory, then the restore arrow back to the GPU.*

These observations motivate offloading. Here is how the Mini-SGLang host cache implements it.

Instead of discarding every evicted prefix, the implementation can save its KV pages in pinned host memory. A shared prefix index tracks whether pages are on the GPU or in the host cache.

When a request returns, the engine restores a compatible host copy before prefill. If no copy survives, it recomputes.

Clean host copies can be retained, avoiding another save when those GPU pages are evicted again. Predictive prefetch can also start restoration before a session returns.

This study keeps twenty-four gibibytes of GPU KV capacity and adds thirty-six gibibytes of host KV capacity. The implemented backing tier is host DRAM; SSD storage remains future work.

## Slide 10 — Fetch-aware ordering discounts host hits

**7:15–8:35 · 80 seconds**

*Read the configurations from top to bottom, then use the three definitions on the right.*

All five bars use the same GPU and host capacities and can restore KV on demand.

The baseline uses first-come, first-served scheduling and LRU eviction. The scheduling-only row replaces first-come, first-served with prefix-aware scheduling: requests needing less new prefill go first. Adapted CacheWise also adds predictive eviction.

The next row keeps CacheWise and adds prefetch. It restores host KV before a session's predicted return, rather than waiting for its next request.

The final row keeps prefetch and adds fetch-aware scheduling. This changes request ordering to recognize host-resident prefixes as cheaper than full recomputation. The score is new prefill tokens plus 0.05 times additional host tokens, with lower cost served first.

The first reported session times are 222, 159, 158, 172, and 136 seconds. Prefetch alone does not improve this observation. The full combination is promising, but the sparse repetitions do not establish statistical significance or isolate every component.

## Slide 11 — Keep the conclusions within the evidence

**8:35–9:20 · 45 seconds**

*Move across the three limitation cards without reading every line.*

Three limits frame these findings.

First, the replication uses synthetic content and selected workloads. One preemption violated the original audit rule. We retained it under a documented post hoc amendment; its extra computation is unmeasured.

Second, the host study uses a separate replay and adds memory capacity. Most configurations have one reported observation, so its policy comparisons are descriptive.

Third, we have no SSD implementation or measurements. An SSD extension would need storage reads and writes, bounded staging, compatibility metadata, and failure handling.

The completed evidence therefore concerns this adapted replication and host-DRAM recovery.

## Slide 12 — Three takeaways

**9:20–10:00 · 40 seconds**

*Deliver each takeaway directly to the audience. Leave a short pause before inviting questions.*

To conclude, there are three takeaways.

First, scheduling and eviction interact. Prefix-aware LRU has the lowest mean in all five replication settings, while predictive eviction changes direction with the scheduler.

Second, saving KV state in host memory can avoid costly recovery, with large reported gains when backing capacity is added.

Third, request ordering should account for where reusable state resides. Fetch-aware scheduling is promising here, while SSD evaluation remains future work.

Thank you. I’m happy to take questions.